

Journal Management System — End-to-End Work Summary

Project: Academic Journal Management System → rebranded to **JGAIR**
Repository path: d:\Journal_management_system\Journal_management_system
Author: Vasudevan Sundaramurthy (vasudevan@mimstechcorp.com)
Report generated: 2026-08-29

1. What this project is

A full-stack, AI-assisted academic journal management platform. It handles the whole editorial pipeline — author submissions, format validation, reviewer suggestion, review collection, editor decisions, notifications, and public journal browsing. Stack: **FastAPI + SQLAlchemy + PostgreSQL** on the backend, **React 18 + TypeScript + Tailwind CSS** on the frontend, an **agent orchestrator** that runs 6 AI agents against the manuscript, and multi-target deploy config for Vercel, Render, Neon, and AWS (Amplify + EC2).

2. Chronological worklog (from git history)

Date	Commit / PR	What it delivered
2026-06-01	Initial commit — Academic Journal Management System	Full first cut of the system: 37,749 lines added across 173 files. Established the entire baseline.
2026-07-16	Prep for free-tier deploy (Vercel + Render + Neon) — PR #1	Trimmed the backend to fit free hosting: removed sentence-transformers/torch (embeddings moved to Voyage AI with Jaccard fallback), removed Celery and replaced it with an in-process InlineTask shim that keeps the task.delay(...) call sites unchanged. Added render.yaml, frontend/vercel.json, and a rewritten README explaining the \$0/month deploy.
2026-07-17	Rebrand to JGAIR + editorial analytics + AI-assisted agents	Renamed the journal brand from IJACR to JGAIR , upgraded 3 AI agents (acknowledgement, format validation, reviewer suggester), expanded the editor portal router (+217 lines) and editor dashboard (+240 lines) with editorial analytics, and swapped the manuscript template. New ai_agent.py service (+252 lines).
2026-07-17	Add GitHub Pages Jekyll workflow	Added .github/workflows/jekyll-gh-pages.yml for GH Pages publishing of the site.
2026-07-17	Terraform config for single-EC2 AWS deploy — PR #2	First infra-as-code: 364-line infra/main.tf, terraform.tfvars.example, and a 123-line user-data.sh.tftpl that bootstraps a single EC2 instance.
2026-07-19	Split-arch Terraform: Amplify frontend + EC2 API	Migrated the infra to a split architecture — static React on AWS Amplify , API on EC2 . Added amplify.yml build spec and rewrote the Terraform to provision both sides.

3. Backend (FastAPI + SQLAlchemy)

3.1 Database models

- bullet **User** — base account with role and MFA fields.
- bullet **Submission** — author's manuscript record; ties into the workflow tasks.
- bullet **Article** — published articles surfaced on the public site.
- bullet **Journal** — journal / issue metadata.
- bullet **Reviewer** — reviewer profile, expertise, availability.
- bullet **Review** — the review record (scores, comments, decision).
- bullet **Notification** — queued and sent notifications.
- bullet **AI Analysis** — stored output from the AI agents.
- bullet **CV Request** — author CV upload / request track.
- bullet **Access Log** — auth and access audit trail.

3.2 Alembic migrations authored

- bullet **3414ab1386b** — initial schema.
- bullet **5f7a9b1c3e2** — add user role and MFA.
- bullet **4e2a1b3d5f7** — add editor portal fields.
- bullet **d39f7a638c9** — add CV requests table.
- bullet **0f1a2b3c4d5** — add author profile + MFA fields.
- bullet **f1a2b3c4d5e6** — extended author profile.

3.3 API routers

- bullet **auth, author_auth, editor_auth** — login/register, OTP, MFA, role gating.
- bullet **submissions** — manuscript upload, status, author view.
- bullet **editor_portal** — editor workspace: queue, assignments, decisions, analytics.
- bullet **editorial** — editorial board and issue-level operations.
- bullet **reviewers** — reviewer directory, expertise, invitations.
- bullet **reviews** — review submission and scoring.
- bullet **articles** — published-article endpoints for the public site.
- bullet **journals** — journal / issue endpoints.
- bullet **ai** — AI-assisted endpoints (summary, plagiarism, recommendations).

3.4 Services (business logic)

- bullet **auth_service, editor_auth, otp_service** — credentials, JWT, OTP.
- bullet **submission_service, review_service, reviewer_service, article_service** — core CRUD and workflow.
- bullet **email_service** (Brevo/SendGrid), **whatsapp_service** (Twilio) — notification channels.
- bullet **notification_service** — unified fan-out layer.

- bullet **pdf_processor** — parses uploaded manuscripts (title, authors, sections).
- bullet **ai_agent** — Anthropic API integration for the agent stack.
- bullet **ai_service** — higher-level AI wrappers (summary, plagiarism-style checks, recommendations).
- bullet **storage_service** — S3-compatible upload (works against AWS S3 or Cloudflare R2).

3.5 AI agents (*agent orchestrator*)

- bullet **agent0_author_profile** — extracts and normalises the author profile.
- bullet **agent1_acknowledgement** — generates the submission acknowledgement.
- bullet **agent2_format_validation** — validates the manuscript against JGAIIR format rules.
- bullet **agent3_reviewer_suggester** — scores reviewers against the paper's topic (Voyage embeddings, Jaccard fallback).
- bullet **agent4_link_generator** — produces signed action links for reviewers.
- bullet **agent5_notification** — dispatches the resulting notifications.
- bullet **orchestrator** — runs the pipeline end-to-end with error handling.

3.6 Background tasks

- bullet Original design: Celery + Redis (`celery_app.py`, `paper_tasks.py`, `notification_tasks.py`).
- bullet Free-tier rework: `InlineTask` shim runs the same task functions on a daemon thread — router-level `.delay(...)` call sites are unchanged, so the switch back to Celery is a one-file swap.
- bullet Tasks covered: new-submission processing, reviewer invitation, deadline reminders, completion notifications.

3.7 Tests

- bullet `test_workflow.py` — 547 lines exercising the end-to-end submission → review → decision flow.
- bullet `test_articles.py`, `test_reviews.py`, `test_ai.py` — per-router unit tests.

4. Frontend (React 18 + TypeScript + Tailwind)

4.1 Pages

- bullet **Dashboard / JournalPage** — landing surfaces for the journal.
- bullet **AboutPage**, **EditorialBoardPage**, **ForAuthorsPage**, **ForReviewersPage** — static/CMS-driven marketing pages.
- bullet **IssuesArchivesPage** + **IssueDetailPage** — issue archive with filtering.
- bullet **ArticlePage** + **ArticleDetail** — public article view.
- bullet **AuthorLoginPage** / **AuthorRegisterPage** / **AuthorDashboard** / **AuthorSubmissionDetail** — author-side flow.
- bullet **SubmitPaper** — 1,027-line multi-step manuscript submission form.
- bullet **EditorLoginPage** / **EditorDashboard** (1,024 lines, editorial analytics added later) — editor workspace.
- bullet **ReviewerPortal** — reviewer landing and review workspace.
- bullet **ReviewPage** — the review form itself.
- bullet **ConsultPartyDashboard** — consultative-party workspace.
- bullet **AllInsightsPage** — surfaces the AI agents' output to the editor.
- bullet **LoginPage** — unified login.

4.2 Reusable components

- bullet **ai/** — AISummary, PlagiarismCheck, RecommendationEngine.
- bullet **articles/** — ArticleDetail, ArticleForm, ArticleList, ArticleSearch, ArticleCard.
- bullet **reviews/** — ReviewForm, ReviewPanel.
- bullet **dashboard/** — HeroSection, JournalMetrics, RecentArticles, NewsAnnouncements, CallForPapers, EditorialBoardPreview, IndexingArchiving, WhyPublishSection, AnnouncementsBanner.
- bullet **layout/** — Header, Footer, Sidebar.
- bullet **common/** — Button, Modal, Loading, JournalLogo, ProtectedEditorRoute.
- bullet **journals/** — JournalCard.

4.3 API + state

- bullet **api/** — client.ts, articles.ts, journals.ts, reviews.ts, ai.ts, auth.ts, editor.js, authorAuth.js.
- bullet **context/AuthContext** and **hooks/** (useAI, useArticles, useAuth) — typed state layer.
- bullet **types/index.ts** — shared TypeScript types.
- bullet **data/issues.ts** — seed issue data used for archive pages.

5. Infrastructure & deployment

5.1 Docker (local dev)

- bullet **backend/Dockerfile** and **frontend/Dockerfile**.
- bullet **docker-compose.yml** — the original multi-service local stack (API, DB, worker, etc.).

5.2 Free-tier stack (\$0/month)

- Frontend on **Vercel** (SPA fallback via `frontend/vercel.json`).
- Backend on **Render**, provisioned by `render.yaml`. Alembic runs on startup.
- Postgres on **Neon** (pooled URL).
- Optional file storage on **Cloudflare R2** (S3-compatible — boto3 unchanged).
- Email through **Brevo** or **SendGrid**; optional **Voyage AI** embeddings.
- Documented trade-offs: ~30 s cold start after 15 min idle, Neon 5-min autosuspend.

5.3 AWS stack (Terraform)

- Single-EC2 blueprint: 364-line `infra/main.tf` + `user-data.sh.tftpl` bootstrap.
- Split-arch upgrade: static frontend on **AWS Amplify** (`amplify.yml`), API on **EC2**. Terraform provisions both sides; user-data script installs the API stack on boot.
- `terraform.tfvars.example` covers all inputs.

5.4 CI / automation

- `github/workflows/jekyll-gh-pages.yml` — publishes docs / static content to GitHub Pages.
- Two PRs merged: PR #1 (free-tier prep) and PR #2 (single-EC2 Terraform).

6. Documentation and manuscript templates

- Docs/ — About.docx, Dashboard.docx, Editor Portal.docx, Editorial_board.docx, Issue_and_Archives.docx, Reviewer.docx, Word_template.docx.
- Backend/app/templates/ — JGAIR manuscript template (renamed from IJACR).
- Frontend/public/Template.zip — downloadable author template bundle.
- README.md rewritten with a full free-tier deploy playbook.

7. Rebrand: IJACR → JGAIR

Renamed the journal identity across the whole codebase in a single commit: templates, logos, page copy, editorial board copy, footer, sidebar, issue data, author/reviewer pages, and the editor dashboard.

8. Headline metrics

Metric	Value
Commits authored	8 (2 via merged PRs)
Initial commit size	173 files, ~37,749 lines added
Backend Python files	~60 (routers, services, models, agents, tasks, tests)
Frontend components + pages	~50 React/TSX files
Alembic migrations	6
AI agents wired into the orchestrator	6 (agent0 – agent5)
Deploy targets configured	Vercel, Render, Neon, Cloudflare R2, AWS Amplify, AWS EC2 (Terraform)
End-to-end workflow test	test_workflow.py — 547 lines

End of report.